



# **Khulna University of Engineering & Technology**

**Department of Electronics and Communication Engineering**

Khulna, Bangladesh

## **CIPHER VAULT – A CLASSICAL CRYPTOGRAPHY SYSTEM**

Course No: CSE 2100

Course Title: Object Oriented Programming Laboratory

Submitted By

**Yeasir Arafat Siju**

roll: 2309058

# CIPHER VAULT

## A Classical Cryptography System Using Object-Oriented Programming in C++

### Abstract

Cryptography is an important technique used to protect information from unauthorized access. This project presents a console-based cryptographic application named **Cipher Vault**, developed using the C++ programming language. The system implements three classical encryption techniques: Caesar Cipher, Atbash Cipher, and ROT13 Cipher. A brute-force attack feature for Caesar Cipher is also included to demonstrate simple cryptanalysis.

The project emphasizes the implementation of Object-Oriented Programming (OOP) concepts such as inheritance, encapsulation, polymorphism, constructors, and modular programming. The program provides a user-friendly menu-driven interface where users can encrypt, decrypt, and analyze ciphertexts efficiently.

### Introduction

With the rapid growth of digital communication, information security has become an essential requirement in modern computing systems. Cryptography is the science of securing data by converting readable information into an unreadable format. It ensures confidentiality and protects sensitive information from unauthorized users.

Classical cryptographic algorithms are simple but important for understanding the basic principles of encryption and decryption. In this project, a cryptographic software named **Cipher Vault** has been developed using C++. The program allows users to perform encryption and decryption using multiple cipher techniques and demonstrates how Object-Oriented Programming concepts can be applied in software development.

## Background Study

Cryptography has been used since ancient times for secure communication. Early civilizations used simple substitution techniques to hide messages from enemies.

Some well-known classical ciphers include:

- **Caesar Cipher**  
A substitution cipher where each letter is shifted by a fixed number of positions in the alphabet.
- **Atbash Cipher**  
A monoalphabetic cipher where letters are reversed, such as  $A \leftrightarrow Z$  and  $B \leftrightarrow Y$ .
- **ROT13 Cipher**  
A special form of Caesar Cipher where the shifting value is always 13.

Although these methods are not secure for modern cybersecurity applications, they are excellent for learning encryption logic, modular arithmetic, and programming implementation techniques.

## Motivation

The main motivation behind this project is to understand how cryptographic algorithms work internally and how Object-Oriented Programming concepts can be implemented practically using C++.

This project also helps in:

- Understanding encryption and decryption mechanisms
- Learning string manipulation techniques
- Applying inheritance and encapsulation
- Building menu-driven applications
- Developing problem-solving skills in programming

## Objectives

The objectives of this project are:

1. To develop a menu-driven cryptographic application using C++
2. To implement Caesar Cipher, Atbash Cipher, and ROT13 algorithms
3. To perform encryption and decryption operations
4. To implement brute-force analysis for Caesar Cipher
5. To apply Object-Oriented Programming concepts in a real application
6. To improve code modularity and reusability

## Methodology

The project was developed using the following methodology:

### Step 1: Program Design

The overall structure of the program was designed using classes and functions.

### Step 2: Cipher Algorithm Implementation

Separate classes were created for Caesar Cipher, Atbash Cipher, and ROT13 Cipher.

### Step 3: OOP Integration

Inheritance and encapsulation were applied to improve code organization and reusability.

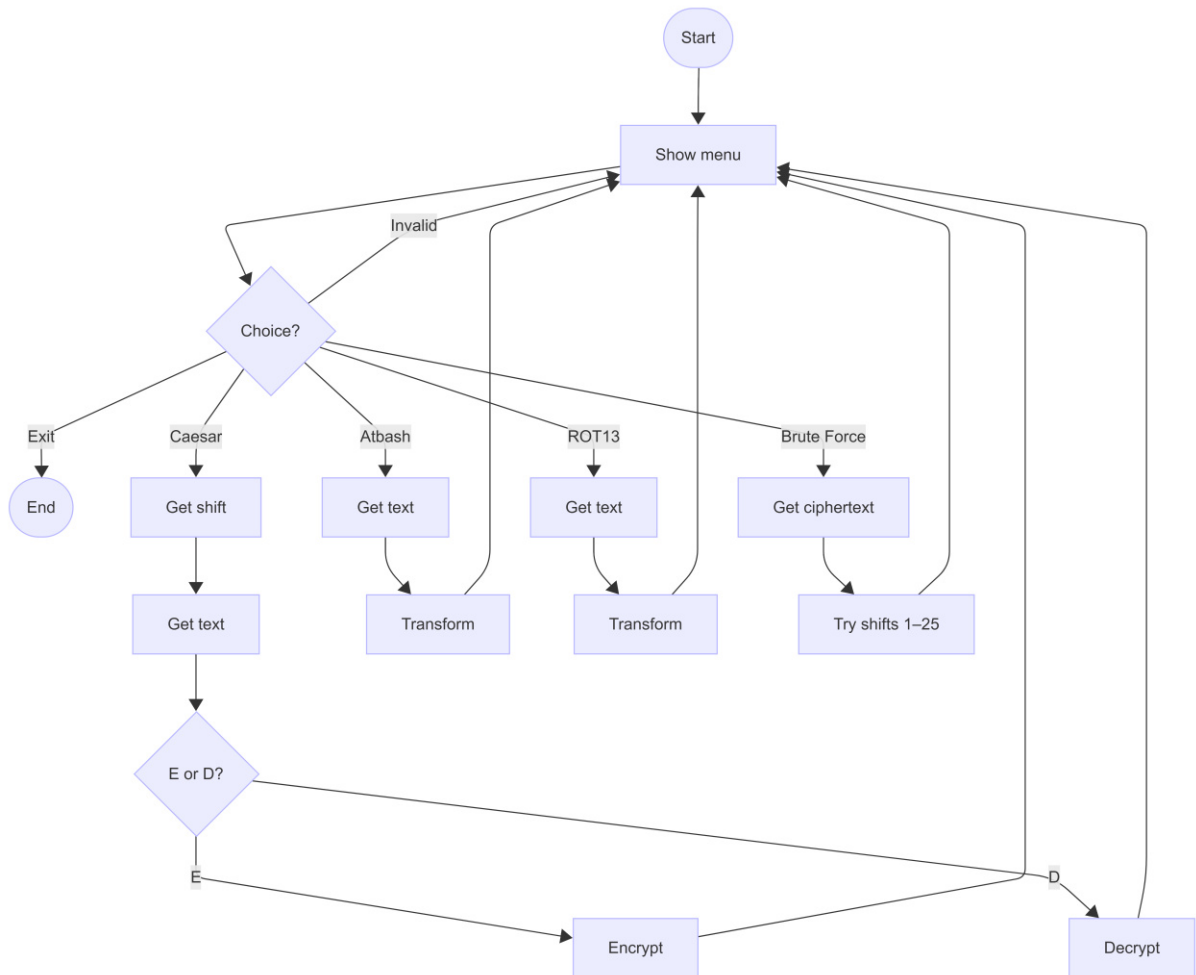
### Step 4: User Interface Development

A menu-driven console interface was implemented for user interaction.

### Step 5: Testing and Validation

Different test cases were applied to verify encryption and decryption correctness.

## System Flow



# Theory of Cryptographic Algorithms

## Caesar Cipher

Caesar Cipher is a substitution cipher where each character is shifted by a fixed number of positions.

### Mathematical Formula

Encryption:  $E(x) = (x + n) \bmod 26$

Decryption:  $D(x) = (x - n) \bmod 26$

Where:

- $x$  = original character position
- $n$  = shift value

```
if (ch >= 'A' && ch <= 'Z')  
    result += (char)((((ch - 'A') + shift) % 26 + 'A'));
```

The program shifts each uppercase letter by the specified shift value using modular arithmetic.

## Atbash Cipher

Atbash Cipher reverses the alphabet mapping.

Example:

- $A \rightarrow Z$
- $B \rightarrow Y$
- $C \rightarrow X$

```
result += (char)('A' + ('Z' - ch));
```

The character is converted into its reverse alphabet equivalent.

## ROT13 Cipher

ROT13 is a Caesar Cipher with a fixed shift value of 13.

Formula:  $ROT13(x) = (x + 13) \bmod 26$

```
result += (char)((ch - 'A' + 13) % 26 + 'A');
```

Each alphabetic character is rotated by 13 positions.

# Object-Oriented Programming Concepts Used

## Class and Object

A class is a user-defined blueprint that groups related data members and member functions together. Objects are instances of classes used to access the functionalities defined inside the class. In this project, different cipher algorithms are implemented using separate classes for better organization and modularity.

```
class CaesarCipher : public Cipher {  
  
};  
  
CaesarCipher cipher(shift);
```

## Inheritance

Inheritance is an important OOP feature that allows one class to acquire the properties and behaviors of another class. It improves code reusability and reduces duplication. In this project, all cipher classes inherit common functionality from the base class Cipher.

```
class CaesarCipher : public Cipher {  
};  
  
class AtbashCipher : public Cipher {  
};  
  
class ROT13 : public Cipher {  
};
```

## Hierarchical Inheritance

Hierarchical inheritance occurs when multiple derived classes inherit from a single base class. This structure allows several classes to share common methods from one parent class. In this project, three cipher classes inherit from the same base class Cipher.

```
class Cipher {  
public:  
    void displayResult(string label, string result) { }  
};  
  
class CaesarCipher : public Cipher {  
};  
class AtbashCipher : public Cipher {  
};  
class ROT13 : public Cipher {  
};
```

## Encapsulation

Encapsulation is the process of combining data and related functions into a single unit while restricting direct access to internal data. This helps protect the data from unauthorized modification. In this project, the shifting value is kept private inside the CaesarCipher class.

```
private:
    int shift;
```

## Constructor

A constructor is a special member function that is automatically executed when an object is created. Constructors are mainly used for initialization purposes. In this project, the constructor initializes and normalizes the shift value for Caesar Cipher.

```
CaesarCipher(int x) {
    shift = ((x % 26) + 26) % 26;
    cout << "Selected shifting value = " << shift << endl;
}
```

## Polymorphism

Polymorphism allows functions with the same name to perform different operations depending on the class implementation. In this project, different cipher classes use functions named encrypt() and decrypt(), but each class performs encryption differently according to its own algorithm.

```
string encrypt(string text) { }

string decrypt(string text) { }

string encrypt(string text) {
    result += (char)('A' + ('Z' - ch));
}

string encrypt(string text) {
    result += (char)((ch - 'A' + 13) % 26 + 'A');
}
```

## Function Reusability

Function reusability is a programming technique where the same function is used multiple times without rewriting code. It improves maintainability and reduces redundancy. In this project, the displayResult() function is reused by all cipher classes through inheritance.

```
void displayResult(string label, string result) {
```



```

cout << "\n-----\n";
cout << label << ":\n";
cout << result << "\n";
}

```

## Modular Programming

Modular programming divides a large program into smaller independent functions or modules. This increases readability, debugging efficiency, and maintainability. In this project, separate functions are used for handling different cipher operations.

```

void handle_caesar()
void handle_atbash()
void handle_ROT13()
void handle_Brute_force()

```

## Abstraction

Abstraction is the process of hiding internal implementation details and showing only essential functionalities to the user. The user interacts with simple menu options without knowing the internal encryption logic.

```

printMenu();
cin >> choice;

switch (choice) {
    case 1:
        handle_caesar();
        break;
}

```

## Access Specifiers

Access specifiers control the visibility and accessibility of class members. C++ mainly uses public, private, and protected access specifiers. In this project, public functions are used for user interaction while private data members protect sensitive internal data.

```

private:
    int shift;

```

## Conditional Statements

Conditional statements allow the program to make decisions based on specific conditions. They control the flow of execution according to user input or program logic.

```
if (op == 'E' || op == 'e')
if (ch >= 'A' && ch <= 'Z')
else if (op == 'D' || op == 'd')
```

## String Handling

String handling is used for storing and manipulating textual data. The project uses C++ string operations extensively for encryption and decryption processes.

```
string text;
getline(cin, text);
string result = "";
```

## Looping Concept

Loops are used to execute a block of code repeatedly until a condition becomes false. In this project, loops are used to process each character of the text and also for brute-force analysis.

```
for (int i = 0; i < (int)text.size(); i++) {
}
for (int s = 1; s <= 25; s++) {
}
```

## Complete Source Code

```
#include <iostream>
#include <string>
#include <limits>
using namespace std;

class Cipher {
public:
    void displayResult(string label, string result) {
        cout << "\n-----\n";
        cout << label << ":\n";
        cout << result << "\n";
        cout << "-----\n" << endl;
    }
};

class CaesarCipher : public Cipher {
private:
    int shift;

public:
    CaesarCipher(int x) {
        shift = ((x % 26) + 26) % 26;
        cout << "Selected shifting value = " << shift << endl;
    }

    string encrypt(string text) {
        string result = "";

        for (int i = 0; i < (int)text.size(); i++) {
            char ch = text[i];

            if (ch >= 'A' && ch <= 'Z')
                result += (char)((((ch - 'A') + shift) % 26 + 'A'));
            else if (ch >= 'a' && ch <= 'z')
                result += (char)((((ch - 'a') + shift) % 26 + 'a'));
            else
                result += ch;
        }

        return result;
    }

    string decrypt(string text) {
        string result = "";

        for (int i = 0; i < (int)text.size(); i++) {
            char ch = text[i];
```

```

        if (ch >= 'A' && ch <= 'Z')
            result += (char)((((ch - 'A') - shift + 26) % 26 + 'A'));
        else if (ch >= 'a' && ch <= 'z')
            result += (char)((((ch - 'a') - shift + 26) % 26 + 'a'));
        else
            result += ch;
    }

    return result;
}

void bruteForce(string cipherText) {
    cout << "\nBrute-Force Results (all 25 shifts):\n";

    for (int s = 1; s <= 25; s++) {
        string result = "";

        for (int i = 0; i < (int)cipherText.size(); i++) {
            char ch = cipherText[i];

            if (ch >= 'A' && ch <= 'Z')
                result += (char)((((ch - 'A') - s + 26) % 26 + 'A'));
            else if (ch >= 'a' && ch <= 'z')
                result += (char)((((ch - 'a') - s + 26) % 26 + 'a'));
            else
                result += ch;
        }

        cout << "  Shift " << (s < 10 ? " " : "") << s << ": " << result
        << endl;
    }
}
};

```

```

class AtbashCipher : public Cipher {
public:
    string encrypt(string text) {
        string result = "";

        for (int i = 0; i < (int)text.size(); i++) {
            char ch = text[i];

            if (ch >= 'A' && ch <= 'Z')
                result += (char)('A' + ('Z' - ch));
            else if (ch >= 'a' && ch <= 'z')
                result += (char)('a' + ('z' - ch));
            else

```

```

        result += ch;
    }

    return result;
}

string decrypt(string text) {
    return encrypt(text);
}

};

class ROT13 : public Cipher {
public:
    string encrypt(string text) {
        string result = "";

        for (int i = 0; i < (int)text.size(); i++) {
            char ch = text[i];

            if (ch >= 'A' && ch <= 'Z')
                result += (char)((ch - 'A' + 13) % 26 + 'A');
            else if (ch >= 'a' && ch <= 'z')
                result += (char)((ch - 'a' + 13) % 26 + 'a');
            else
                result += ch;
        }

        return result;
    }

    string decrypt(string text) {
        return encrypt(text);
    }
};

void printMenu() {
    cout << "\n===== CIPHER VAULT =====\n";
    cout << " 1. Caesar Cipher\n";
    cout << " 2. Atbash Cipher\n";
    cout << " 3. ROT13 Cipher\n";
    cout << " 4. Brute-Force Caesar\n";
    cout << " 5. Exit\n";
    cout << " Enter your choice: ";
}

string getText(string prompt) {
    string text;
    cout << prompt;

```

```

        getline(cin, text);
        return text;
    }
    char getOperation() {
        char op;
        cout << "\nOperation - Type 'E' for encrypt or 'D' for decrypt: ";
        cin >> op;
        cin.ignore();
        return op;
    }

    void handle_caesar() {
        cout << "\n---- CAESAR CIPHER ----\n";

        int shift;

        while (true) {
            cout << "Enter shift value (1-25): ";
            cin >> shift;

            if (cin.fail()) {
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                cout << " *** Invalid input. Please enter a number. *** \n";
            }
            else {
                cin.ignore();
                break;
            }
        }

        CaesarCipher cipher(shift);

        string text = getText("Enter text: ");
        char op = getOperation();

        if (op == 'E' || op == 'e')
            cipher.displayResult("Encrypted", cipher.encrypt(text));
        else if (op == 'D' || op == 'd')
            cipher.displayResult("Decrypted", cipher.decrypt(text));
        else
            cout << " *** Invalid operation. ***" << endl;
    }

    void handle_atbash() {
        cout << "\n---- ATBASH CIPHER ----\n";
        cout << "(A<->Z, B<->Y ... encrypt = decrypt)\n";

        AtbashCipher cipher;
    }

```

```

        cin.ignore();

        string text = getText("Enter text: ");

        cipher.displayResult("Result", cipher.encrypt(text));
    }
    void handle_ROT13() {
        cout << "\n---- ROT13 CIPHER ----\n";
        cout << "(Fixed shift of 13 ... encrypt = decrypt)\n";

        ROT13 cipher;
        cin.ignore();

        string text = getText("Enter text: ");

        cipher.displayResult("Result", cipher.encrypt(text));
    }
    void handle_Brute_force() {
        cout << "\n---- BRUTE-FORCE CAESAR ----\n";

        cin.ignore();

        string text = getText("Enter ciphertext to brute-force: ");

        CaesarCipher cipher(0);
        cipher.bruteForce(text);
    }

    int main() {
        int choice;

        cout << "\n Welcome!\n";

        do {
            printMenu();
            cin >> choice;

            if (cin.fail()) {
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                cout << "\n *** Invalid input. Please enter a number between 1-
5. *** \n\n";
                continue;
            }
            switch (choice) {
                case 1:
                    handle_caesar();
                    break;

```

```

        case 2:
            handle_atbash();
            break;
        case 3:
            handle_ROT13();
            break;
        case 4:
            handle_Brute_force();
            break;
        case 5:
            cout << "\n..... Goodbye!\n\n";
            break;
        default:
            cout << "\n ** Invalid choice. Please enter 1-5. ** \n\n";
    }
}
while (choice != 5);
return 0;
}

```

## Sample Output

Caesar Cipher:

```

PS C:\Users\arafa\OneDrive\Desktop\CSE> cd "c:\Users\arafa\OneDrive\Desktop\CSE\"
; if ($?) { g++ Caesar_Cipher.cpp -o Caesar_Cipher } ; if ($?) { .\Caesar_Cipher
}

Welcome!

===== CIPHER VAULT =====
1. Caesar Cipher
2. Atbash Cipher
3. ROT13 Cipher
4. Brute-Force Caesar
5. Exit
Enter your choice: 1

---- CAESAR CIPHER ----
Enter shift value (1-25): 5
Selected shifting value = 5
Enter text: hello everyone

Operation - Type 'E' for encrypt or 'D' for decrypt: e

-----
Encrypted:
mjqqt jajwdsj
-----

```



## Atbash Cipher:

```
===== CIPHER VAULT =====
  1. Caesar Cipher
  2. Atbash Cipher
  3. ROT13 Cipher
  4. Brute-Force Caesar
  5. Exit
Enter your choice: 2

---- ATBASH CIPHER ----
(A<->Z, B<->Y ... encrypt = decrypt)
Enter text: I am not a robo.

-----
Result:
R zn mlg z ilyl.
-----
```

## ROT13:

```
===== CIPHER VAULT =====
  1. Caesar Cipher
  2. Atbash Cipher
  3. ROT13 Cipher
  4. Brute-Force Caesar
  5. Exit
Enter your choice: 3

---- ROT13 CIPHER ----
(Fixed shift of 13 ... encrypt = decrypt)
Enter text: A quick brown fox

-----
Result:
N dhvpx oebja sbk
-----
```

## Brute-Force Caesar:

```
===== CIPHER VAULT =====
1. Caesar Cipher
2. Atbash Cipher
3. ROT13 Cipher
4. Brute-Force Caesar
5. Exit
Enter your choice: 4

---- BRUTE-FORCE CAESAR ----
Enter ciphertext to brute-force: jumps over a lazy dog
Selected shifting value = 0

Brute-Force Results (all 25 shifts):
Shift 1: itlor nudq z kzyx cnf
Shift 2: hsknq mtcp y jyxw bme
Shift 3: grjmp lsbo x ixwv ald
Shift 4: fqilo kran w hwvu zkc
Shift 5: ephkn jqzm v gvut yjb
Shift 6: dogjm ipyl u futs xia
Shift 7: cnfil hoxk t etsr whz
Shift 8: bmehk gnwj s dsrq vgy
Shift 9: aldgj fmvi r crqp ufx
Shift 10: zkcfi eluh q bqpo tew
Shift 11: yjbeh dktg p apon sdv
Shift 12: xiadg cjsf o zonm rcu
Shift 13: whzcf bire n ynml qbt
Shift 14: vgybe ahqd m xmlk pas
Shift 15: ufxad zgpc l wlkj ozr
Shift 16: tewzc yfob k vkji nyq
Shift 17: sdvyb xena j ujih mxp
Shift 18: rcuxa wdmz i tihg lwo
Shift 19: qbtwz vcly h shgf kvn
Shift 20: pasvy ubkx g rgfe jum
Shift 21: ozrux tajw f qfed itl
Shift 22: nyqtw sziv e pedc hsk
Shift 23: mxpsv ryhu d odcg grj
Shift 24: lworu qxgt c ncba fqi
Shift 25: kvnqt pwfs b mbaz eph
```

## Observations

- All cipher algorithms produced correct outputs.
- The brute-force system successfully generated all possible Caesar Cipher combinations.
- The menu-driven interface improved usability.
- Input validation prevented invalid operations.

## **Advantages of the Project**

- Simple and user-friendly interface
- Demonstrates basic cryptography concepts
- Shows practical implementation of OOP
- Easy to modify and extend
- Supports multiple cipher techniques

## **Conclusion**

The “Cipher Vault” project successfully demonstrates the implementation of classical cryptographic techniques using C++ and Object-Oriented Programming principles. The application performs encryption, decryption, and brute-force analysis efficiently while maintaining code modularity and readability.

This project helped in understanding the practical implementation of inheritance, encapsulation, polymorphism, constructors, loops, conditional statements, and modular programming. It also provided valuable knowledge about classical cryptographic systems and secure data transformation techniques.